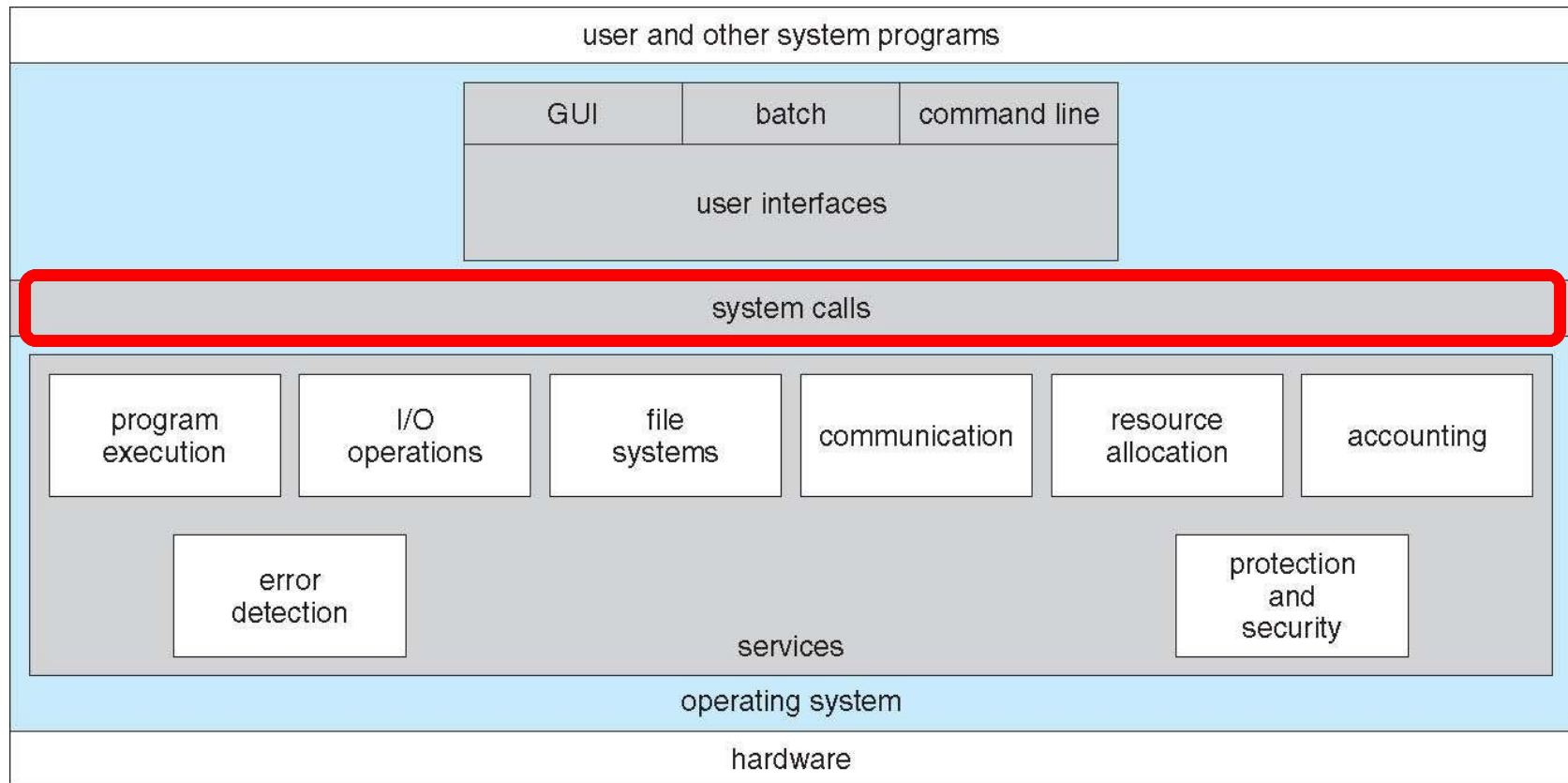


Operating Systems

CS2006

Lecture 3

System Calls

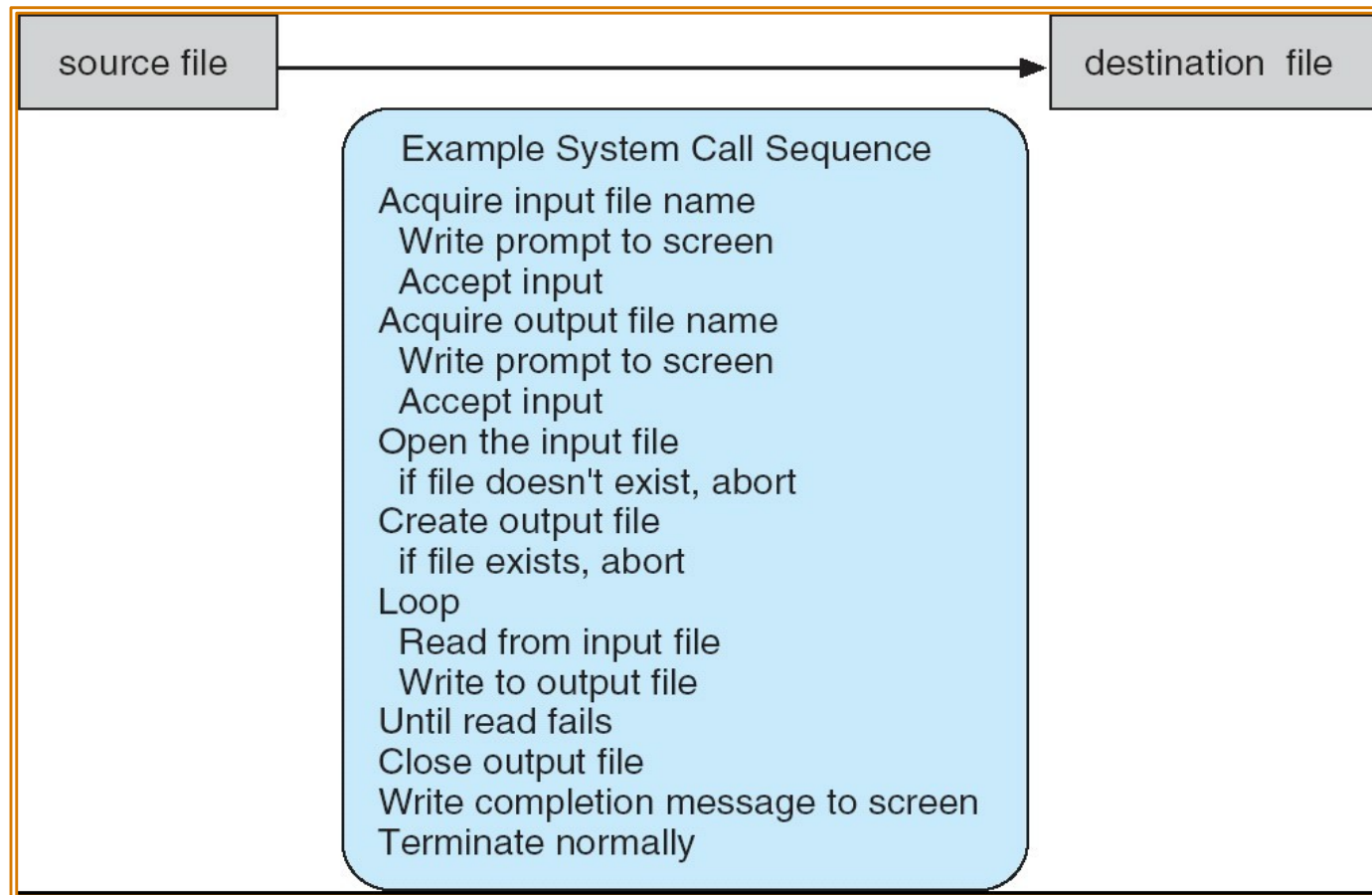


System Calls

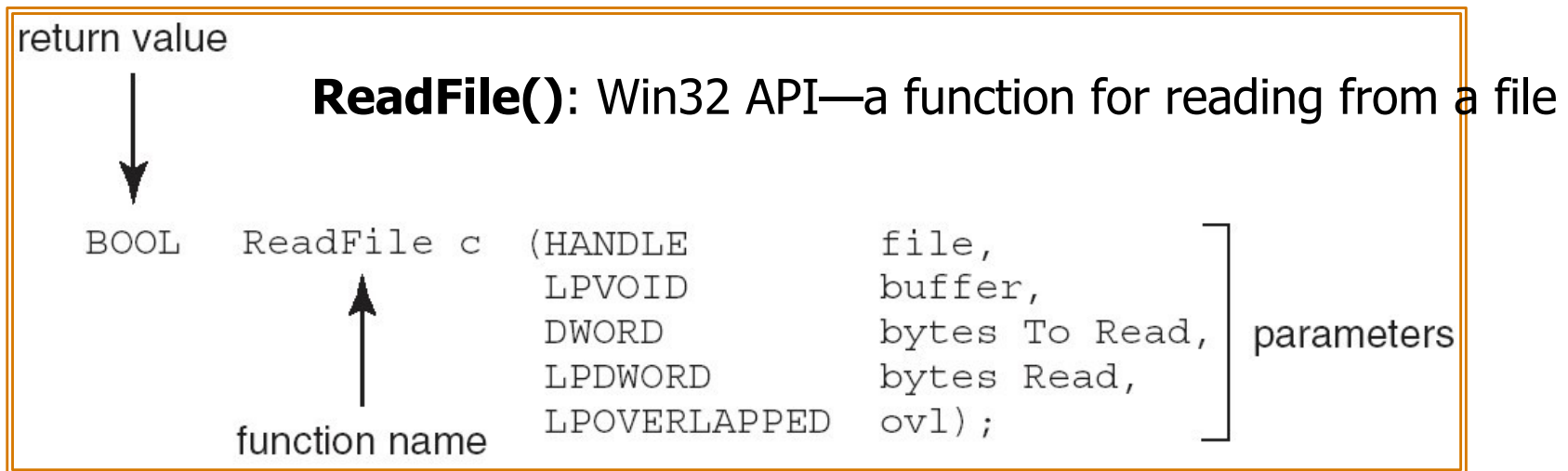
- Programming interface to the services provided by OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Program Interface (API)** rather than direct system call use
- Three most common APIs are **Win32 API** for Windows, **POSIX API** for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and **Java API** for the Java virtual machine (JVM)
- **Why use APIs rather than system calls?**

Example of System Calls

- System call sequence to copy the contents of one file to another file

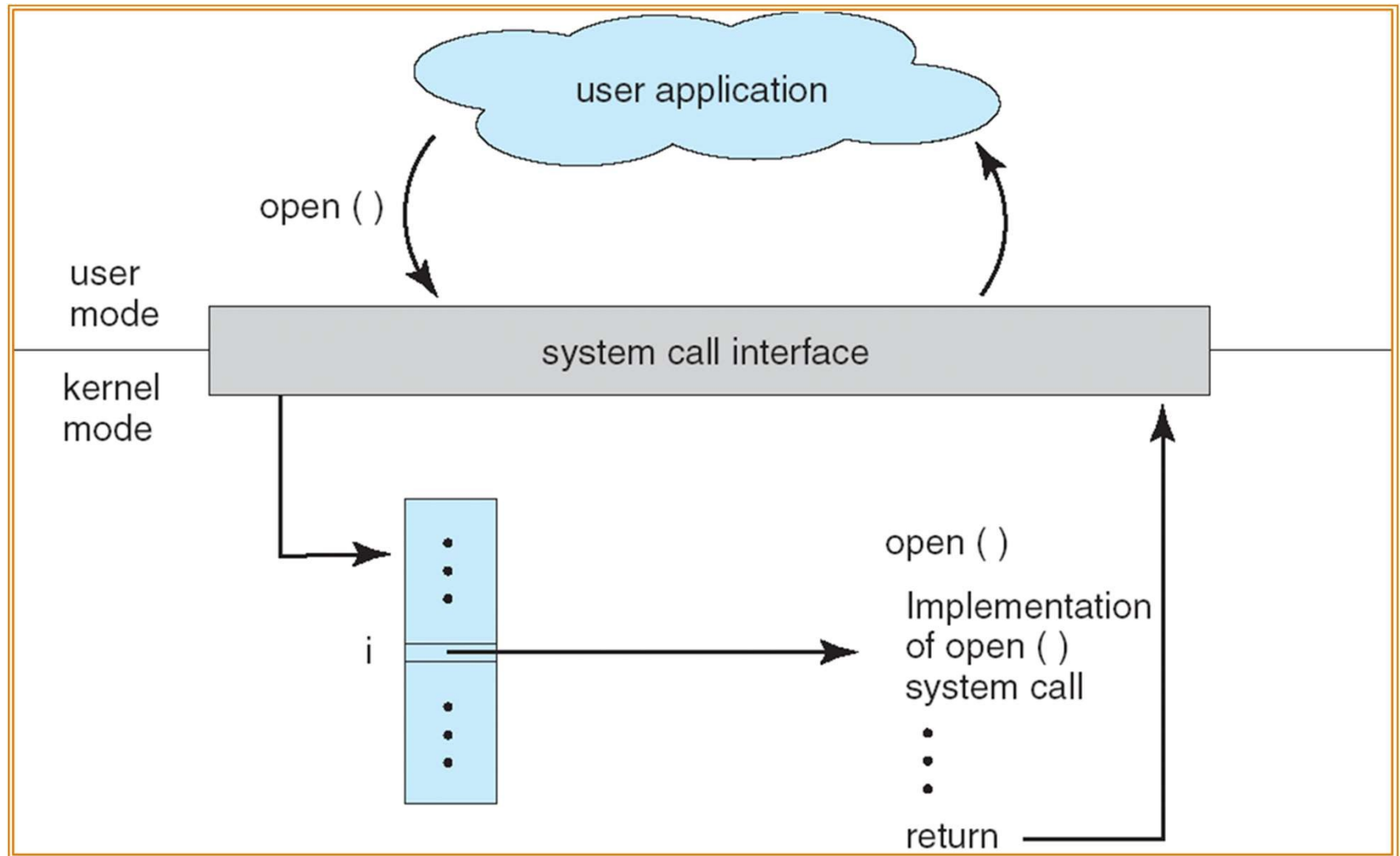


Example of Standard API



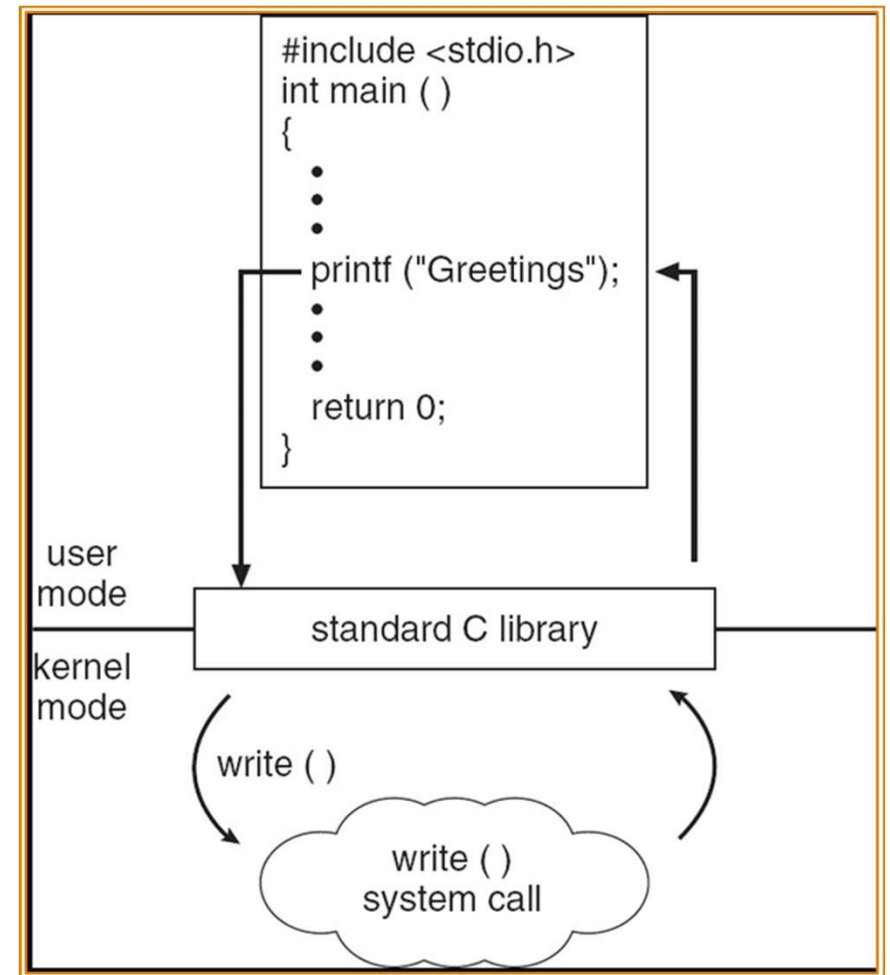
- A description of the parameters passed to ReadFile()
 - HANDLE file—the internal handle of the file to be read
 - LPVOID buffer---A buffer where the data will be read into and written from
 - DWORD bytesToRead—the number of bytes to be read into the buffer
 - LPDWORD bytesRead—the number of bytes read during the last read
 - LPOVERLAPPED ovl—indicates if overlapped I/O is being used

API – System Call – OS Relationship



Standard C Library Example

- C program invoking printf() library call, which calls write() system call



Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

OS Structure

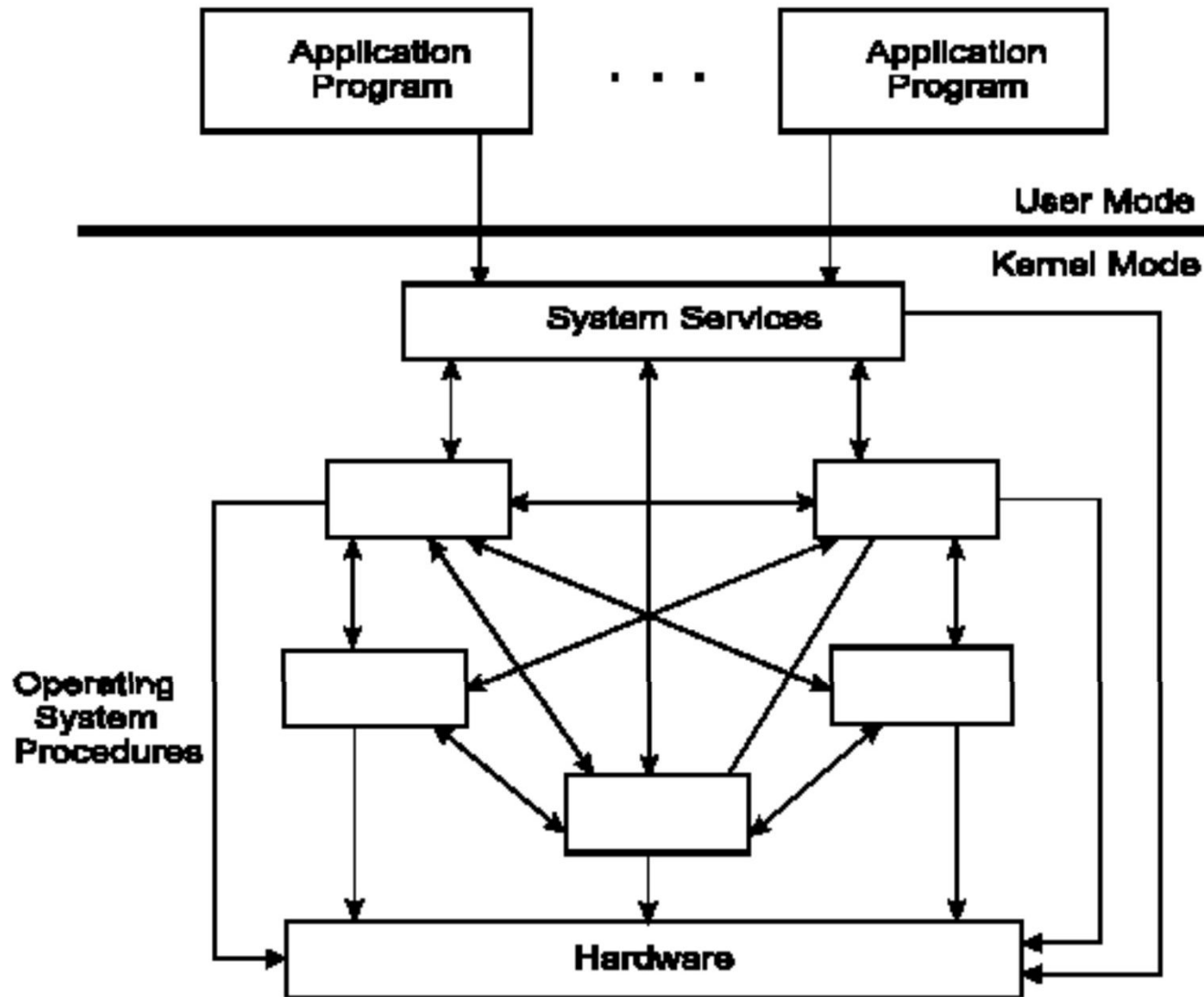
Operating System Design and Implementation

- Affected by choice of hardware, type of system
- *User* goals and *System* goals
 - *User goals* – operating system should be convenient to use, easy to learn, reliable, safe, secure, and fast
 - *System goals* – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error free, secure, and efficient
- **Important principle is separation**
 - *Policy*: What will be done?
 - *Mechanism*: How to do it?
 - The separation of policy from mechanism is a very important principle, it allows maximum *flexibility* if policy decisions are to be changed later

Operating Systems Structures

- Structure/Organization/Layout of OSs:
 - Monolithic (one unstructured program)
 - Layered
 - Microkernel
 - Virtual Machines

1. Monolithic Operating System



Monolithic OS – Basic

Structure

- Application programs that invoke the requested system services.
- A set of system services that carry out the operating system procedures/calls.
- A set of utility procedures that help the system services.

MS-DOS System Structure

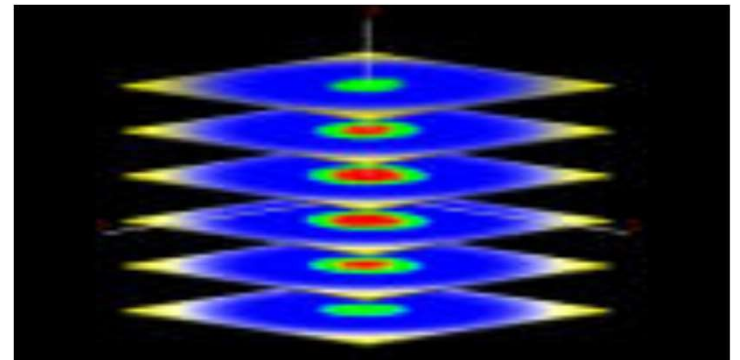
- MS-DOS – written to provide functionality in the least space:
 - not divided into modules (monolithic).
 - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated.



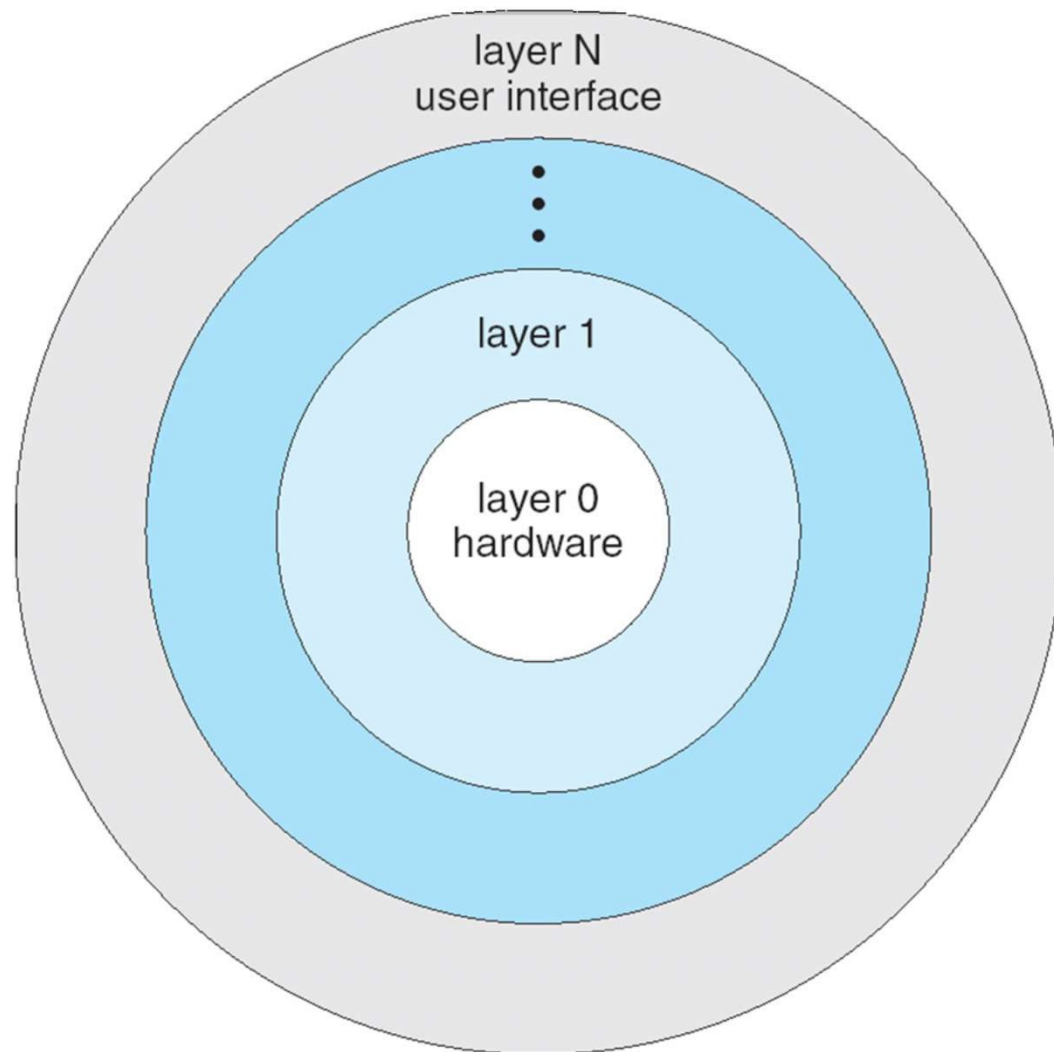
```
C:\WINNT\System32\cmd.exe
05/13/00 10:18a <DIR> My Music
08/04/00 03:32p <DIR> palm
05/17/00 10:44a 104 PMig.Log
05/17/00 10:52a 836 PMig0.Log
05/17/00 10:49a 291 PMig01.Log
03/11/01 01:18p <DIR> Program Files
05/10/00 05:14p <DIR> service pack 5
12/04/00 10:20a <DIR> TBDiscount
11/26/98 06:11p 766 Tele.ico
03/11/01 04:43p <DIR> TEMP
07/11/00 10:36p <DIR> test.txt
01/28/01 12:24a <DIR> try
08/17/98 11:09p 169,472 US-Oper.exe
09/16/98 08:01p 409,600 WB32.EXE
03/11/01 01:08p <DIR> Windows Update Setup Files
03/11/01 03:23p <DIR> WINNT
05/13/98 11:12p 201,728 WinRAR.exe
03/11/01 03:38p 3,237 winzip.log
05/30/00 10:39a 12 WZT1
11/14/00 03:31p 83,644 xWave.wav
01/19/00 01:04p 79,360 æÿÿÿ_æÿÿÿ.doc
41 File(s) 1,353,864 bytes
1,725,181,440 bytes free
C:\>
```

2. Layered Approach

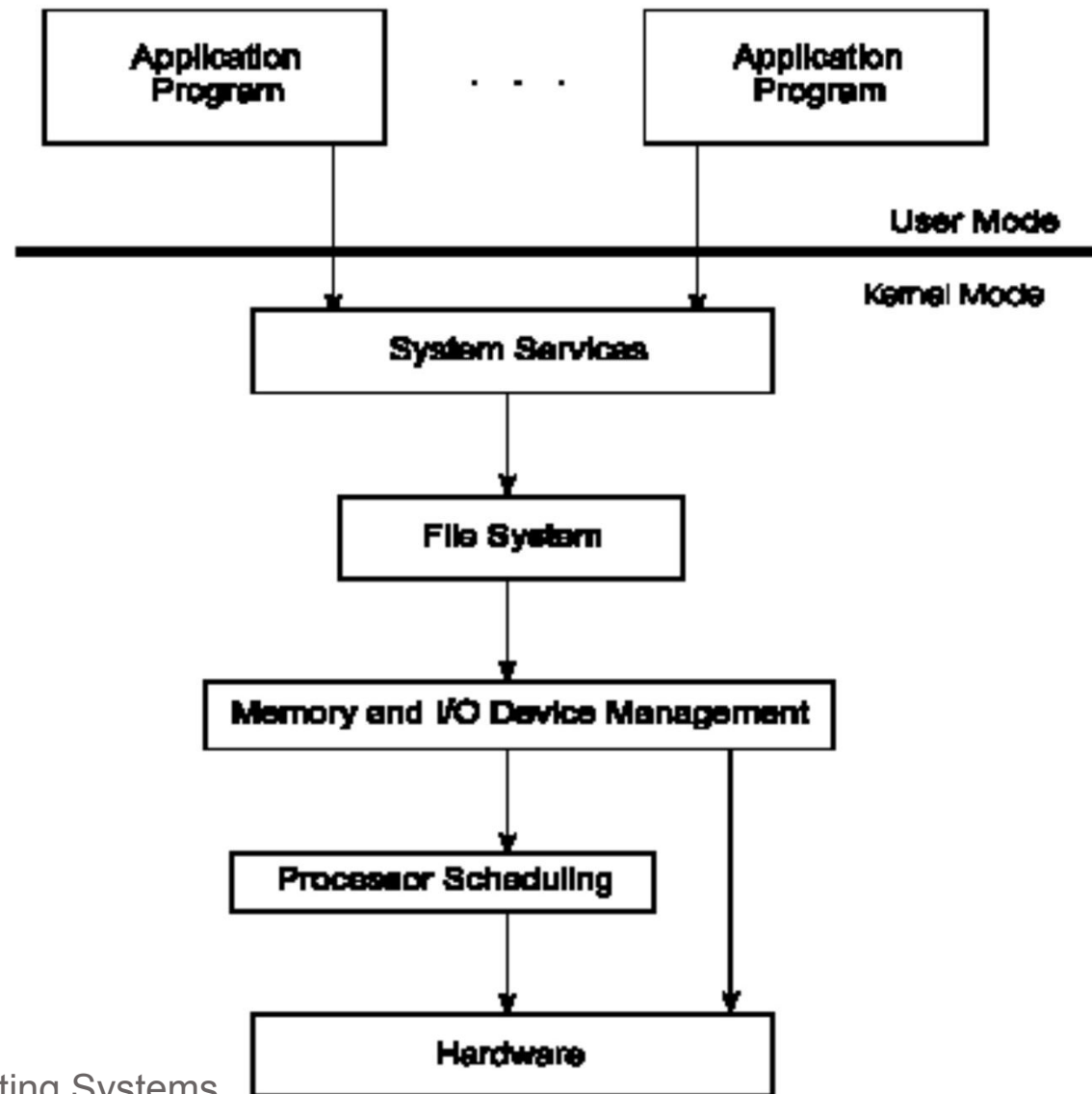
- The operating system is divided into a number of layers (levels), each built on top of lower layers
- The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers



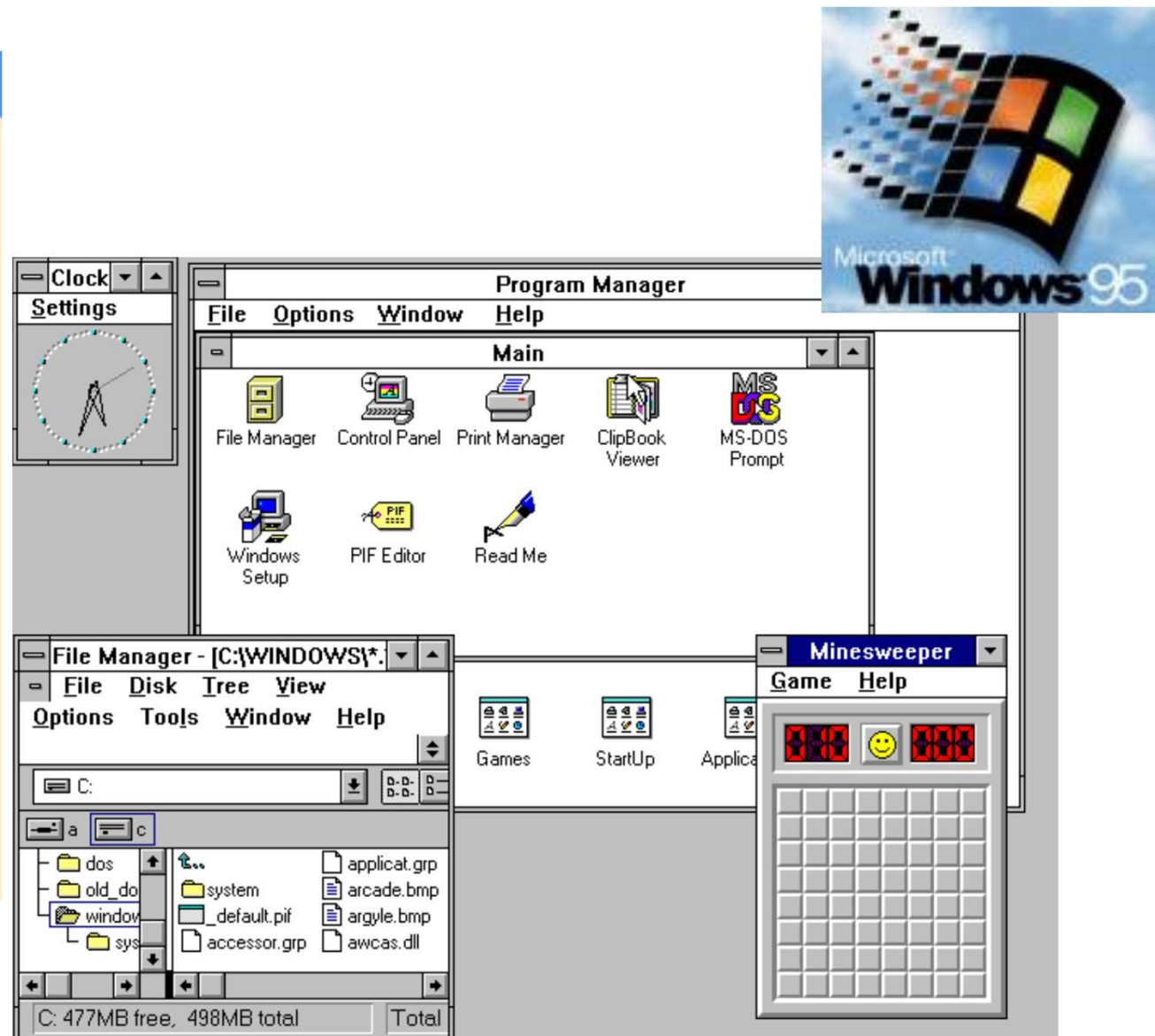
Layered Operating System



Operating System Layers



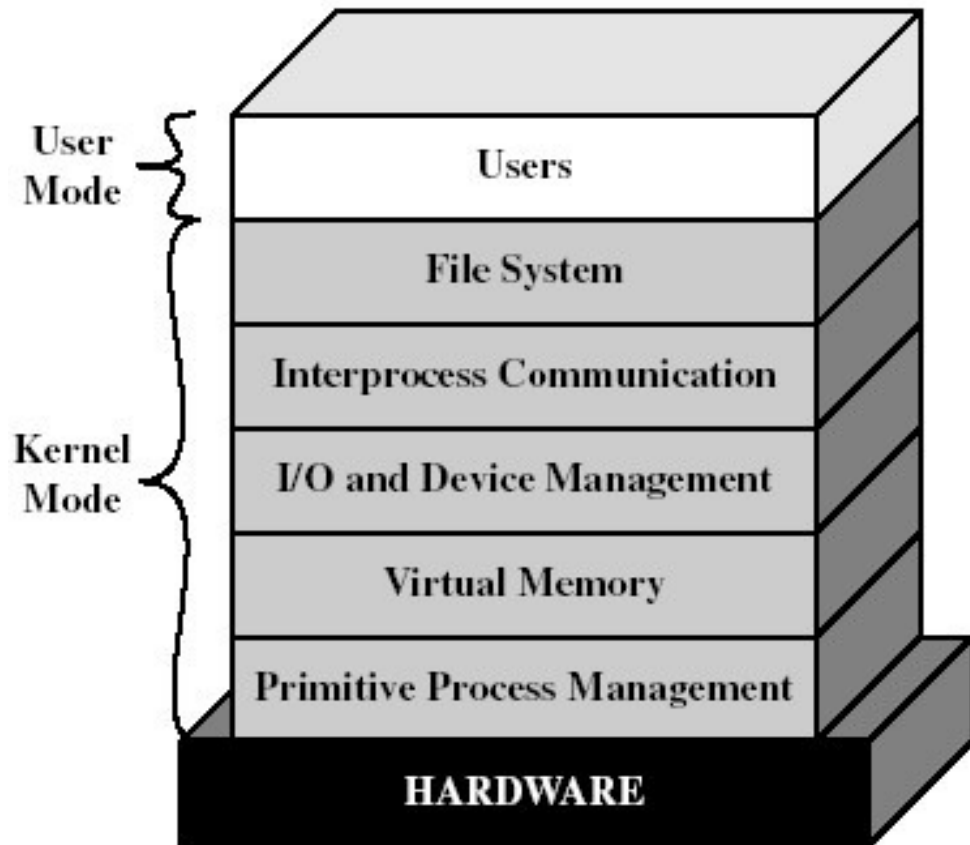
Older Windows System Layers



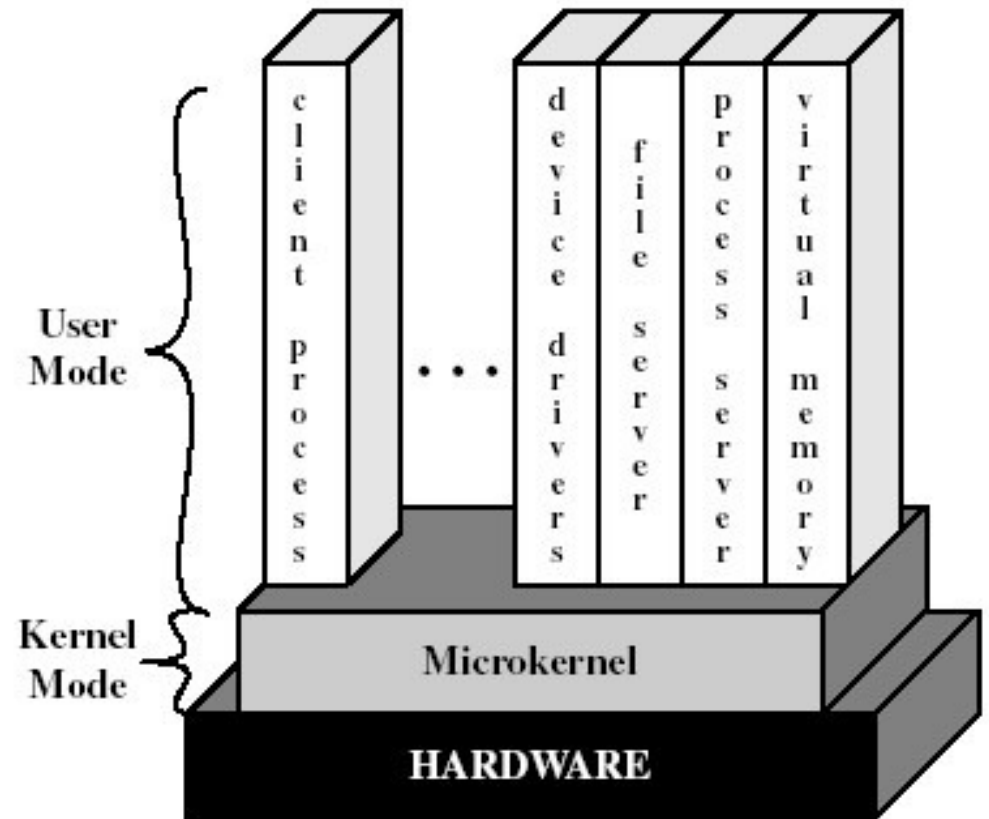
3. Microkernel System Structure

- Move as much functionality as possible from the kernel into “user” space.
- Only a few essential functions in the kernel:
 - primitive memory management (address space)
 - I/O and interrupt management
 - Inter-Process Communication (IPC)
 - basic scheduling
- Other OS services are provided by processes running in user mode (vertical servers):
 - device drivers, file system, virtual memory...

Layered vs. Microkernel Architecture



(a) Layered kernel



(b) Microkernel

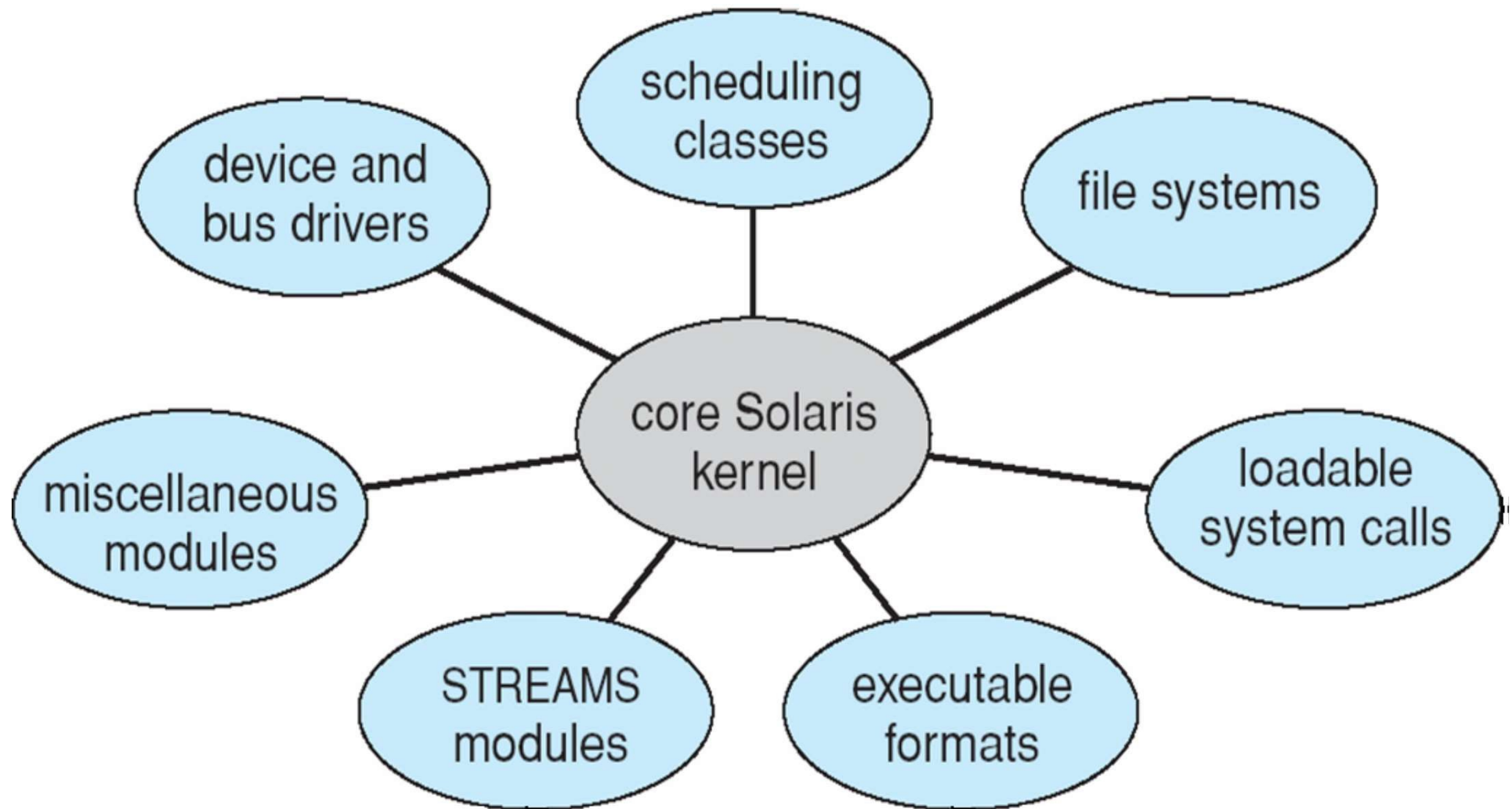
Benefits of a Microkernel Organization

- Extensibility/Reliability
 - modular design
 - easier to extend a microkernel
 - more reliable (less code is running in kernel mode)
 - more secure (less code to be validated in kernel)
 - small microkernel can be rigorously tested
- Portability
 - changes needed to port the system to a new processor is done in the microkernel, not in the other services.

Kernel Modules

- Most modern operating systems implement kernel modules
 - Uses object-oriented approach
 - Each core component is separate
 - Each talks to the others over known interfaces
 - Each is loadable as needed within the kernel
- Overall, similar to layers but more flexible

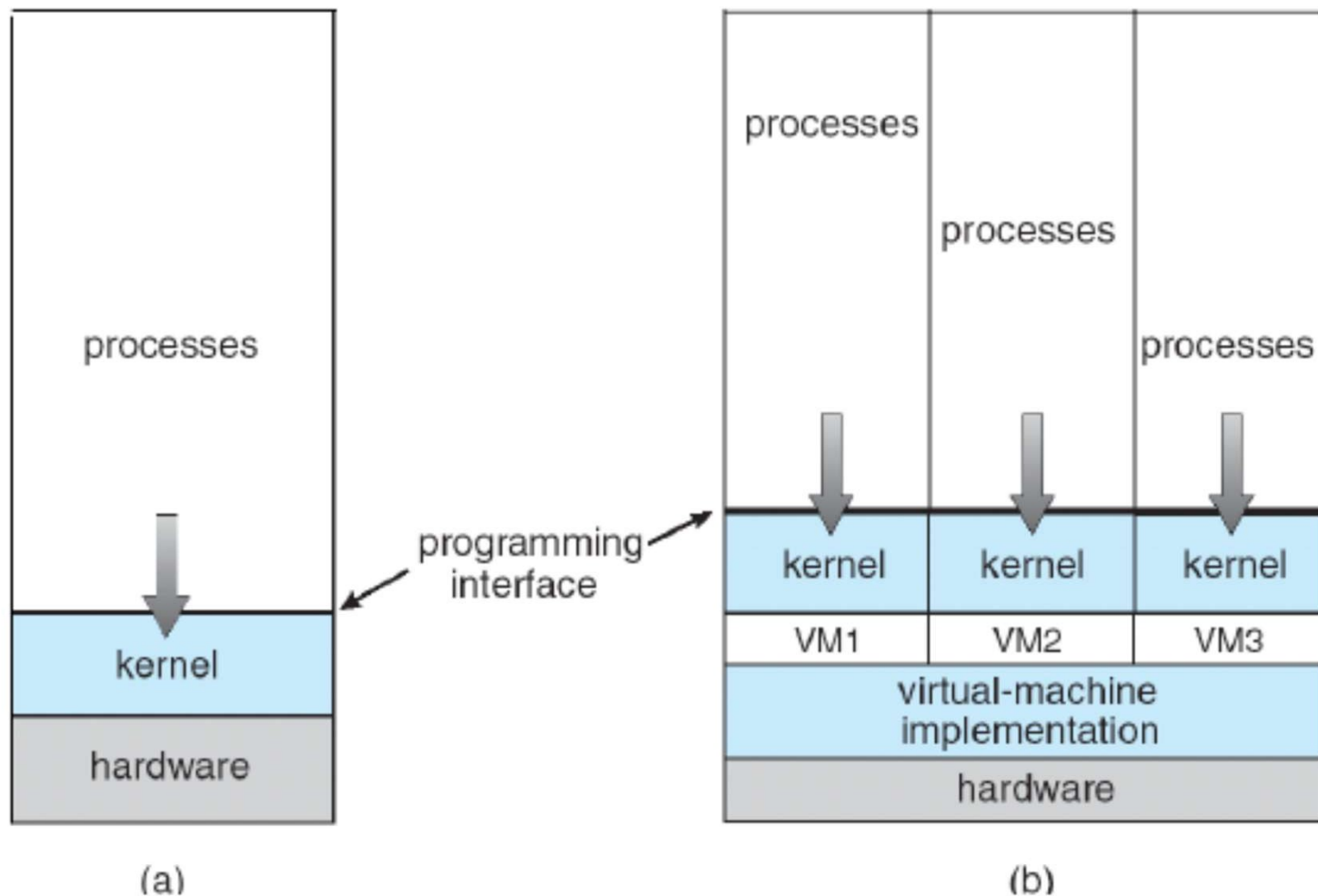
Solaris Modular Approach



4. Virtual Machines

- A **virtual machine** takes the layered approach to its logical next step. It treats hardware and the operating system kernel as though they were all hardware
- A virtual machine provides an interface identical to the underlying bare hardware
- The operating system **host** creates the illusion that a process has its own processor (and virtual memory)
- Each **guest** provided with a (virtual) copy of underlying computer

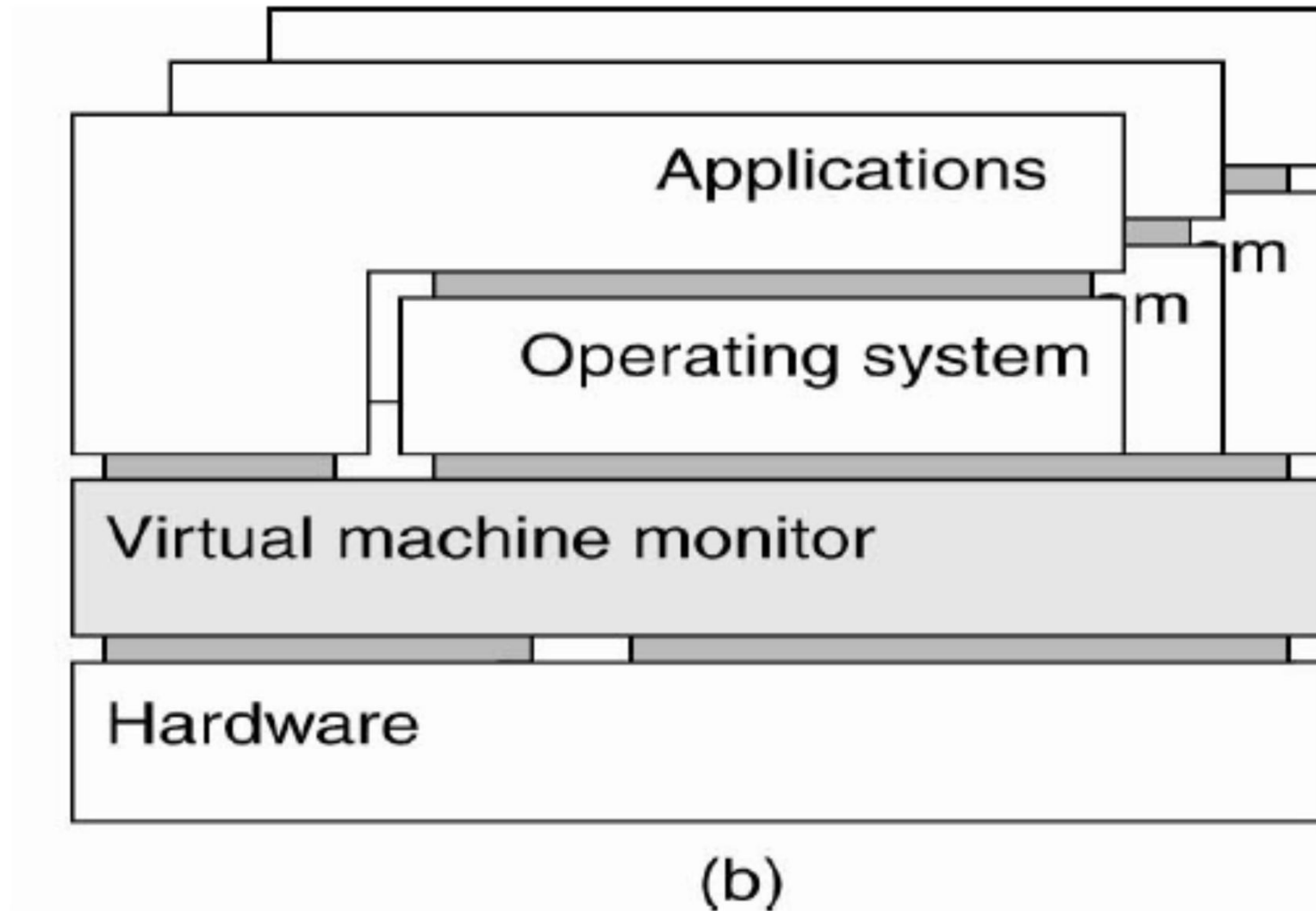
Virtual Machines (Cont.)



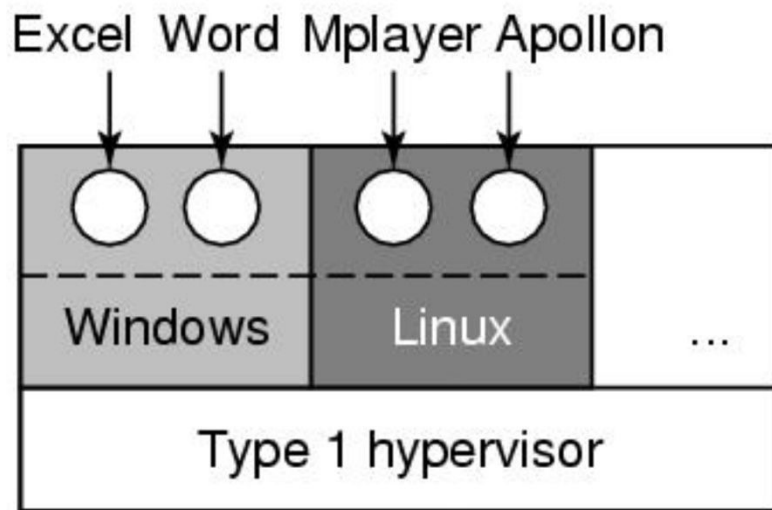
(a) Non-virtual machine

(b) virtual machine

Hypervisor / VMM

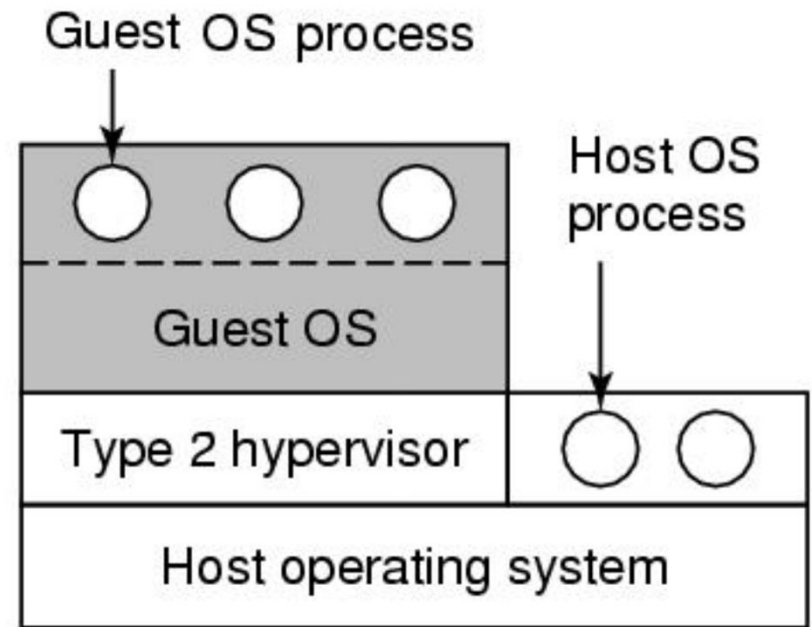


Types of Hypervisors



(a)

(a) A type 1
hypervisor



(b)

(b) A type 2
hypervisor